Raccolta dei Requisiti con le User Stories

Docente: Michele Marchesi

A.A. 2025-26

marchesi@unica.it

Introduzione (1/2)

- Le metodologie agili si basano sull'osservazione che né il cliente né lo sviluppatore hanno una conoscenza completa del sistema nelle prime fasi del processo di sviluppo. **Essi imparano man mano che il sistema evolve**
- È fondamentale adottare delle pratiche che permettano ad entrambi di poter sfruttare la propria crescente competenza durante il processo di sviluppo
- *The hardest part of the software task is arriving at a complete and consistent specification, and much of the essence of building a program is in fact the debugging of the specification* (Fred Brooks, 1987)

Introduzione (2/2)

- In un processo tradizionale, i requisiti devono essere definiti dal cliente in modo molto preciso nella prima fase di analisi:
  - Apportare delle modifiche man mano che il processo di svilup-po avanza diventa sempre più difficile e oneroso
  - I requisiti sono documentati in modo rigoroso e formale
- Le MA lasciano al cliente la libertà di decidere in qualunque momento quali funzionalità realizzare e di richiedere modifiche delle funzionalità esistenti
- La parola *Agile* si riferisce alla possibilità di far fronte al conti-nuo cambiamento dei requisiti
- Identificare i requisiti in modo rigoroso è lungo e costoso, e spesso poco utile perché i requisiti possono cambiare molto velocemente

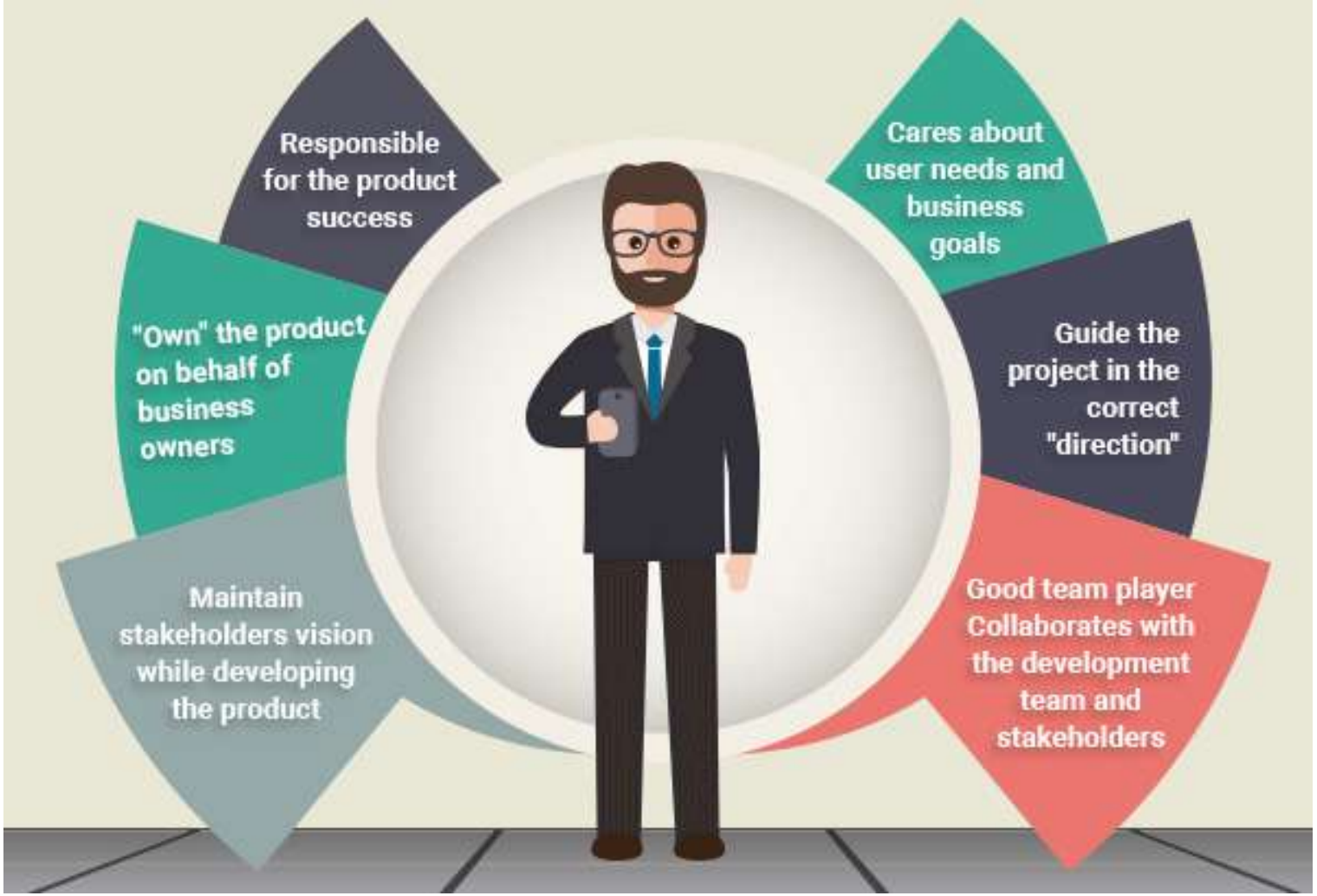
Requisiti Agili (1/3)

- Nell'approccio agile, i requisiti sono espressi ad alto livello, con frasi brevi (chiamate *Features* o *User Stories*) che **contengono solo le informazioni più importanti**
- Nessuna forma di documentazione cerimoniosa, formale e *voluminosa*
- L'insieme delle features governa lo sviluppo **iterativo-incrementale**:
  - **Ogni iterazione implementa un dato numero di features**, scelte dal cliente

Requisiti Agili (2/3)

- Massima interazione tra gli *stakeholder*
  - Permette agli sviluppatori di capire i dettagli di cosa il cliente vuole veramente
  - Nel caso migliore: ***real customer involvement*** (il cliente è disponibile durante tutto il processo di sviluppo)
  - In alternativa: si usa un ***proxy*** del cliente *on-site*, cioè una persona che conosce il problema, ed agisce per conto del cliente (il **Product Owner** o PO)

Il Product Owner



Requisiti Agili (3/3)

- Rilasci frequenti del software:
  - Solo se il cliente può usare una versione funzionante del sistema può fornire *feedback* e specificare meglio i requisiti
  - Lo sviluppo si basa su iterazioni (tipicamente di una/due settimane)
  - Almeno ogni tre mesi viene rilasciata una versione funzionante del software

Esprimere i Requisiti Agili

- **Feature** o Caratteristica - MMF (Scrum e FDD)
  - MMF: **Minimum Marketable Feature**
  - E' una breve frase che descrive cosa il cliente vuole (funzionalità o requisito non funzionale)
  - La chiarezza è essenziale
    - Esempio: *gli oggetti all'asta possono essere aggiunti, modificati e cancellati*
- **User Story** (XP)
  - Breve descrizione dell'interazione di un utente o sistema esterno col sistema
- Entrambe guidano lo sviluppo!

User Stories (1/3)

- Una User Story, o Storia, è una breve descrizione del comportamento del sistema, dal punto di vista dell'utente (brevi Casi d'Uso)
- E' scritta in linguaggio naturale
- Per ogni funzionalità principale del sistema, deve essere scritta almeno una storia
- Ogni Storia deve **fornire valore al cliente**
- Sono scritte dal cliente o dal Product Owner, in genere con l'assistenza degli sviluppatori

User Stories (2/3)

- Sono scritte su *index card* (schede)*:*
  - schede in formato A6 (10x15 cm)
- Dovrebbero essere **indipendenti** le une dalle altre, per favorire lo scheduling
- Sono **stimate** dagli sviluppatori, cioè gli sviluppatori devono essere in grado di prevedere quanto tempo impiegheranno per realizzarla
- L'elenco delle US da implementare si chiama ***backlog*** (lavoro arretrato)

User Stories (3/3)

- **Appartengono al cliente** (o al **PO**), che ne definisce la priorità
- Ogni storia deve essere testabile, questo evita che si scrivano storie ambigue
- Non devono essere esaustive, ma sono ***occasioni per un incontro col cliente o col PO***
- In pratica, **devono essere spiegate** dal cliente (o dal PO) agli sviluppatori, che possono rivolgersi al cliente anche in seguito, per ulteriori chiarimenti

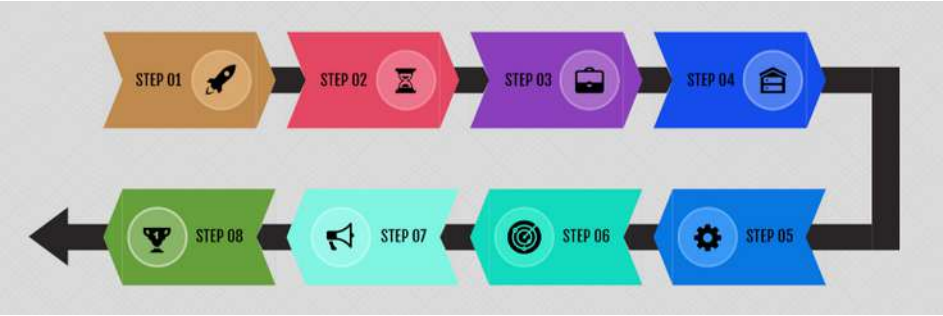
User Stories vs Casi d'Uso

- Entrambi descrivono funzionalità del sistema
- Entrambi sono **incrementi** dello sviluppo incrementale, e possono *guidare le iterazioni*
- User Stories **NON** è un altro modo di chiamare i CdU
- Le User Stories sono dei documenti **molto più semplici** dei casi d'uso, e non sono esaustive
- Le storie sono scritte dal cliente o dal PO
- I casi d'uso sono scritti dagli analisti, sentendo le esigenze del cliente
- I casi d'uso riportano una descrizione più completa del requisito relativo

Raccogliere le Storie (1/2)

- In un processo Agile, i requisiti si raccolgono durante tutta la durata del progetto
- La maggior parte dei requisiti, però, è raccolta all'inizio del progetto
- Nella prima fase i requisiti sono raccolti attraverso una serie di riunioni, alle quali partecipano sia il cliente che gli sviluppatori

Raccogliere le Storie (2/2)



- Due approcci
  - *Goal-oriented*
    - Punto di partenza: obiettivo del sistema
    - Quali passi sono necessari per raggiungere tale obiettivo?
    - I passi sono scritti nelle user stories
  - *Scattergun*
    - Non viene data nessuna struttura alle riunioni
    - Le storie vengono scritte sulla base di ciò che emerge dalla riunione

INVEST!

- Una US deve essere:
  - **I**ndipendente
  - **N**egoziabile (col cliente)
  - di **V**alore
  - (**E**)stimabile
  - della giusta dimensione (**S**ize)
  - **T**estabile

Indipendente

- Le US dovrebbero essere implementate in modo indipendente
- Precedenze e altri vincoli vanno minimizzati
- Un esempio di dipendenza:
  - Prima si implementa la US "*Registrazione*"
  - Poi si possono implementare le US "*Login*", "*Logout*", ecc.

Negoziabile (con il cliente o il PO)

- Le US **non** sono:
  - Contratti scritti
  - Requisiti assoluti
- ***Non si devono includere troppi dettagli***
- **Troppi dettagli** danno l'impressione di:
  - Falsa precisione o completezza
  - Nessun bisogno di comunicare ancora
- Le US si devono poter “aggiustare”, trattando col cliente o col Product Owner quanta parte della US va implementata

Di Valore

- Ogni US deve **dare valore** al cliente o all'utente
- Una volta eseguita, l'utente deve essere più “ricco”:
  - con maggiori informazioni
  - il sistema ha fatto qualcosa di utile per l'utente
  - talora, con un guadagno monetario reale
- **Valore per l'utente**: *tutto ciò che l'utente è disposto a pagare per avere!*

Stimabile

- Le US sono usate per pianificare lo sviluppo del sistema
- Una US non è stimabile se:
  - È troppo grande (è un'**Epica**)
  - Gli sviluppatori non conoscono il dominio
  - Gli sviluppatori non hanno le competenze tecniche
- Se una US non è stimabile, siamo in una condizione di **rischio** per lo sviluppo del sistema

Testabile

- Ogni US dovrebbe essere dotata di uno o più *Test di Accettazione (TA)*
- Un TA è un test funzionale che verifica la corretta implementazione della US
- I TA dovrebbero essere scritti in codice, ed eseguibili in modo automatico tutte le volte che serve
- Moderni framework come *Selenium* si possono usare per simulare interazioni tra utenti e pagine Web

Un possibile schema per le US

- In qualità di *[ruolo utente]*
- voglio *[azione]*
- in modo che/da *[motivazione]*

In qualità di ***"frequent flyer"***
voglio ***esaminare il conto delle mie miglia***
in modo da ***verificare se manca qualche volo***

In qualità di ***cliente***
voglio ***cancellare una prenotazione alberghiera***
in modo da ***ottenere il rimborso***

Altro possibile schema per US e relativi TA

- US:
  - As a [type of user], I want to [perform some task] so that I can [achieve some goal]
- TA:
  - Given that [some context], when [some action is carried out], then [a set of observable outcomes should occur].

US: Come ciclista, voglio vedere i percorsi ciclabili fatti dei miei amici in modo che possa unirmi a loro nei giri in bici.

TA: Dato che un ciclista vuole pedalare con gli amici, quando controlla la mappa, deve vedere i percorsi degli altri ciclisti della sua rete sociale.

US troppo grandi (Epiche)

- Se una US è troppo grande, ad es. ci vuole troppo tempo per realizzarla, si chiama **epica**
- In tal caso, va scomposta in modo che:
  - Tutto sia espresso come US
  - Ogni US sia realizzabile al massimo in pochi giorni
  - Le US siano indipendenti tra loro

Esempio di US dal progetto C3

* Cost of Living Allowance (COLA)
  * EEs may receive COLA. Some unions pay a fixed $ amount per pay. Some pay a percentage of base. Amount or percentage varies by union and location. COLA rates change at specified times for different unions.
* Each EE receives COLA in each check

Il TA relativo (dal progetto C3)

* Acceptance Test
  * Provide EEs from each union and from no union. Pay each EE for three pay periods, test whether COLA amount properly calculated.
  * Determine whether COLA amount properly displayed on check or EFT stub.

Esempio: User Story testuale

Title: Login		
Acceptance Test: LoginTest	Priority: 2	Story Points: 2
A user has to input his/her nickname and password to login the system. If the combination of the nickname and password is not correct, an error message will show, and the user has to input the nickname and password again. Personal page and bid functionality are available only when the user logs in successfully. Administrative functionality is available only when the user with administrative privileges logs in successfully.		

User Story secondo schema standard

Title: Login		
Acceptance Test: UserLoginTest	Priority: 2	Story Points: 2
In qualità di utente voglio effettuare il login digitando user e password, in modo da entrare nel sistema		

Title: Login Amministratore		
Acceptance Test: AdminLoginTest	Priority: 3	Story Points: 1
In qualità di amministratore voglio effettuare il login digitando user e password, in modo da entrare nel sistema coi privilegi di amministratore		

Elementi delle schede

- ***Titolo***: due o tre parole; dovrebbe essere un verbo attivo al presente (simile al nome di un Caso d'Uso)
- ***Acceptance Test***: Lista dei test di accettazione associati alla storia
- ***Priorità***: **Il cliente decide** quali storie devono essere implementate per prime. Solitamente si usa una scala 1-2-3, dove 1 è il max
- ***Story Points***: Sforzo di sviluppo (*Effort*) stimato **dagli sviluppatori**
- ***Descrizione***: alcune frasi (o frase standard) che esprimono i passi da eseguire per raggiungere l'obiettivo

Stima di una storia (1/2)

- E' preferibile stimare la dimensione delle storie usando come unità di misura i ***punti-storia***
- Un punto-storia corrisponde spesso (ma non sempre) a una giornata di lavoro
- Altri considerano come unità di misura la mezza giornata, altri l'ora di lavoro...
- Se il programmatore stima che la storia richiederà più di due settimane di lavoro (ad es. più di 10 punti), questa è un'**Epica**, ed egli dovrà richiede-re al cliente di suddividere la storia in parti più semplici (*split*)

Stima di una storia (2/2)

- La stima può essere semplice nel caso in cui si disponga di storie simili già implementate
- Quando non si è in grado di fare una stima ragionevole della storia si fanno degli esperimenti, noti come ***spike solution*** (prototipizzazione rapida):
  - Obiettivo delle spike solution: sperimentare soluzioni e avere un'idea più chiara dei tempi necessari per implementare una storia
  - Il codice prodotto viene poi di solito scartato
  - È l'approccio agile alla ***gestione del rischio***

Test di Accettazione

- Sono scritti dal cliente o dal PO; per ogni storia può essere definito anche più di un test di accettazione
- Sono automatizzati dal team di sviluppo e devono essere ripetibili
- Verificano che la funzionalità richiesta dal cliente sia stata implementata correttamente
- Una storia si ritiene completata quando vengono superati con successo tutti i test di accettazione ad essa relativi

Altri Requisiti

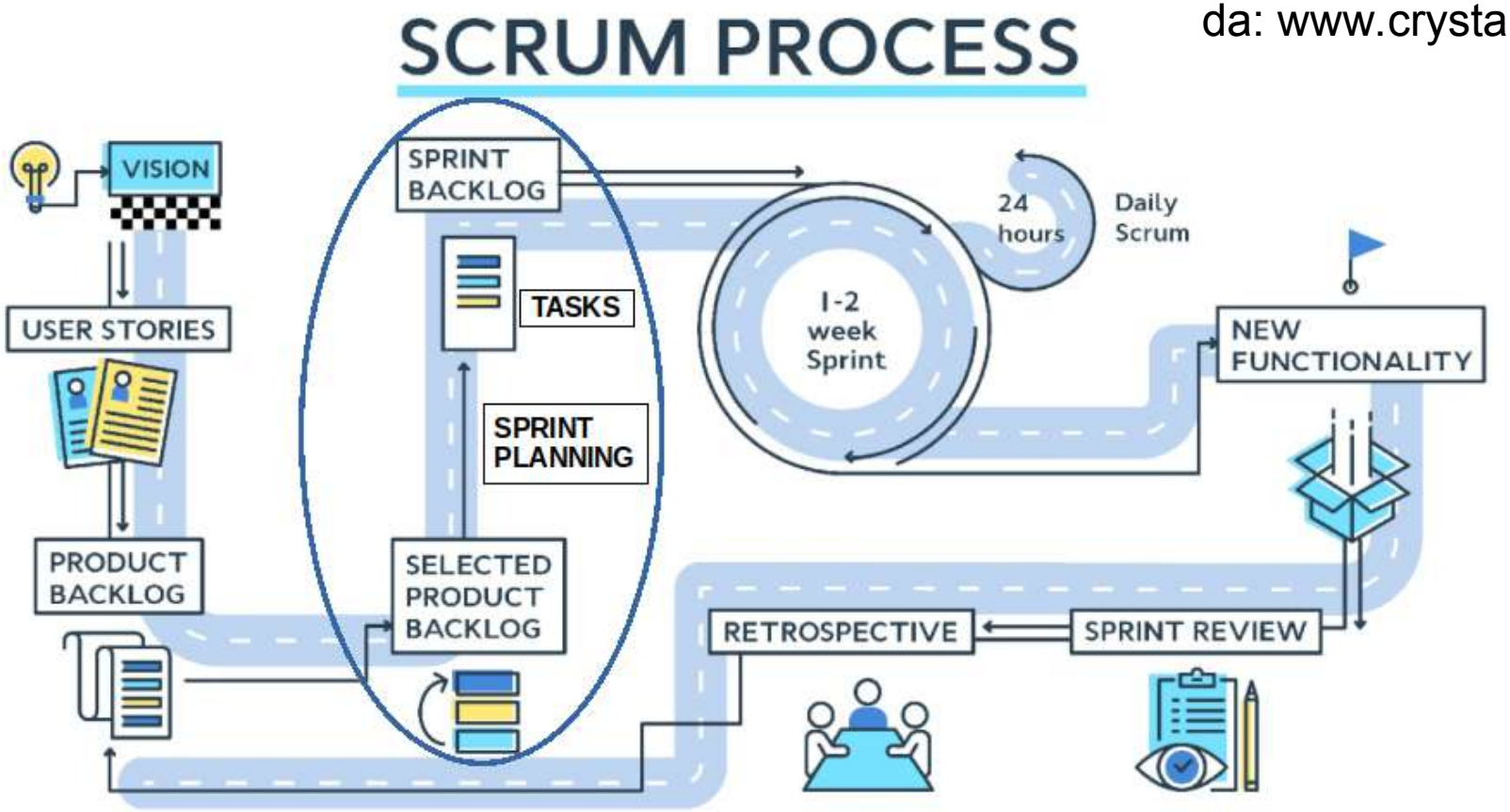
- I casi d'uso e le storie, come è stato detto, documentano i requisiti funzionali
- I requisiti non-funzionali e i vincoli non sono facilmente esprimibili come user stories
- Nelle user stories possono comunque essere esplicitati una serie di requisiti non funzionali e di vincoli
  - Esempio: *Il sistema deve rispondere a determinate interrogazioni in meno di 300 ms*.

US e processo agile

- Nelle MA lo sviluppo è **guidato dalle US** (o *features*)
  - I requisiti del sistema sono espressi come US, ciascuna valutata in *punti storia* e con una priorità
  - Tali US sono inserite in un elenco (*Backlog* di progetto)
  - Il progetto procede per rilasci (*Release*), composti da iterazioni brevi (*Sprint*) ciascuna con uno Sprint Backlog
  - Lo Sprint Backlog è un sottoinsieme di US da implementare nello Sprint, scelte in una riunione all'inizio dello Sprint
  - In questa riunione, il cliente (o il PO) spiega in dettaglio al team le US da implementare nello Sprint, ed è disponibile durante lo sviluppo per ulteriori spiegazioni
  - All fine di ogni Sprint, il cliente esegue i test di accettazione su ogni US per verificare se è stata implementata correttamente

Scrum Process

da: www.crystalloids.com/



Il processo agile sarà spiegato in dettaglio nella sezione relativa del corso